

# FINAL PROJECT REPORT: SKYTRACKER

Course: Web Application Construction | Date: September 2026

Author: Artyom Vorontsov | Degree Program: Computer Science

| Component          | Specification / Technology                       |
|--------------------|--------------------------------------------------|
| Backend Framework  | NestJS (TypeScript, Node.js)                     |
| Frontend Framework | React (TypeScript) with Leaflet Mapping          |
| Database System    | PostgreSQL with TypeORM ORM                      |
| Stylesheet Tech    | SASS (.scss) with Mixins & Glassmorphic UI       |
| External Open API  | OpenSky Network Live Aircraft Telemetry REST API |
| Deployment         | Docker, Docker Compose, Nginx Multi-stage build  |

## 1. Executive Summary & Project Objectives

SkyTracker is a dynamic web application built to fulfill the final project requirements for the *Web Application Construction Course* at TSI. The system tracks live airborne aircraft in real-time by integrating with open aviation REST APIs (OpenSky Network), processing state vectors on a NestJS backend, and visualizing dynamic aircraft positions, flight headings, and telemetry metrics on a high-performance React frontend.

## 2. Technical Architecture & Implementation

The application adheres strictly to modern two-tier web architecture separation:

- **Backend Tier (NestJS):** Implements modular controllers (`FlightController``) and services (`FlightService``) handling asynchronous HTTP data fetch from OpenSky API, data transformation, telemetry aggregation, and PostgreSQL database synchronization using TypeORM entities (`FlightEntity``, `WatchlistEntity``). OpenAPI Swagger documentation is embedded at `/api/docs``.
- **Frontend Tier (React & TypeScript):** Built using functional components and React hooks for real-time state management. Integrated Leaflet map rendering uses dynamic SVG aircraft markers rotated live based on heading angles.
- **Persistence (PostgreSQL):** Relational database storage for flight telemetry logs, user-selected aircraft watchlists, and priority alert settings.

## 3. Fulfillment of Course Guidelines & Dynamic Elements

The system features 5 distinct dynamic elements as mandated by the course specifications:

| Dynamic Element           | Description & Implementation                                                                                                          |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1. Dynamic Radar Map      | Interactive Leaflet canvas rendering moving aircraft markers rotated live dynamically according to telemetry heading.                 |
| 2. Telemetry Inspector    | Interactive detailed side-panel inspecting live airspeed, altitude, vertical rate, and ICAO registration upon selecting any aircraft. |
| 3. Airspace Aggregator    | Live statistical dashboard computing average airspace speed, total airborne vs. grounded aircraft, and peak altitude.                 |
| 4. Multi-Attribute Filter | Real-time responsive search bar and toggle filters (e.g., Airborne Only) instantly pruning the flight vector dataset.                 |
| 5. Pinned Watchlist       | Dynamic bookmarking feature allowing users to pin critical aircraft to a watchlist persisted via REST API to PostgreSQL.              |

## 4. Stylesheet Techniques & Preprocessing

Styling was crafted using **SASS (SCSS)** preprocessor features. Key design techniques include:

- **Glassmorphism UI:** Created using `@mixin glass-panel` leveraging backdrop filters (`backdrop-filter: blur(16px)`), subtle translucent background gradients, and glowing borders.
- **Nested Scoping & Variables:** Organized color schemes (`$accent-color`, `$bg-dark`) ensuring dark-mode visual hierarchy.

## 5. Deployment & Containerization Setup

To enable production hosting, the project includes multi-stage Docker builds and a master `docker-compose.yml` file. The frontend is compiled to static assets served via an optimized Nginx web server container, while NestJS and PostgreSQL run in isolated network bridges.

## 6. Conclusion & Results

All requirements outlined in the TSI Web Application Construction course guidelines have been fully implemented. The application demonstrates robust software architecture, clean REST API contracts, modern styling, dynamic user interactivity, and production container readiness.